

ICPC Bootcamp Day 5 Summary

Based on `day5_slides.pptx`

Session Summary

Day 5 introduces two families of mathematical contest tools: number theory and computational geometry. The number theory portion begins with the contest motivation. Direct mathematical algorithms are often too slow, so students use divisibility facts, prime structure, and modular arithmetic to reduce work from linear or quadratic scans to $\mathcal{O}(\sqrt{N})$, $\mathcal{O}(\log N)$, or near-linear preprocessing. The slides also remind students to watch integer overflow in fixed-width languages and to remember that Python's large integers are convenient but can still be slower for very large values.

Prime numbers are the first concrete topic. Trial division checks divisibility only up to $\sqrt{N}$, because any factor larger than $\sqrt{N}$ must pair with a smaller factor. To generate many primes, the Sieve of Eratosthenes marks composite multiples in a boolean array and runs in roughly $\mathcal{O}(M \log \log M)$ for all primes up to M . The same prime list supports optimized factorization: divide N only by primes up to $\sqrt{N}$, record repeated factors, and if the remaining value is greater than 1, treat it as the final prime factor.

The modified sieve slide shows that a sieve can store more than true/false primality. A smallest-prime-factor array supports fast repeated factorization. Similar sieve-style preprocessing can compute values such as Euler's totient function or counts of distinct prime factors. The GCD and LCM slide then introduces the Euclidean algorithm and the identity $\text{lcm}(A, B) = A \cdot (B / \text{gcd}(A, B))$, with division before multiplication to reduce overflow risk.

Modular arithmetic closes the number theory section. Modulo operations keep intermediate values small, are often required by problem statements, and appear naturally in cyclic indexing and hashing. Students should remember that addition, subtraction, and multiplication work well under a modulus, but modular division requires an inverse rather than ordinary division.

The geometry portion begins with floating-point error. Because decimal values cannot always be represented exactly, code should avoid direct equality comparisons on floating-point numbers. The slides recommend using tolerance-based comparisons, such as Python's `math.isclose`, when checking equality or boundary cases.

Basic geometry objects are then introduced. Points are the building blocks and are usually represented by a class, struct, tuple, or complex number, with methods for equality, ordering, distance, and rotation. Lines can be represented in standard form $Ax + By + C = 0$, which handles vertical lines better than slope-intercept form. Circles are represented by center and radius, with area, circumference, and point-position checks. Triangles require the triangle inequality and support area formulas such as base-height and Heron's formula. Quadrilaterals are usually handled by decomposing them into triangles rather than memorizing many special cases.

The final slides cover polygon algorithms. A polygon is stored as an ordered list of points, usually counterclockwise, and many implementations duplicate the first point at the end to simplify edge iteration. The shoelace formula computes polygon area in linear time. Convexity can be tested by checking that consecutive triples turn consistently using cross products. Point-in-polygon uses ray casting or winding

logic, with special care for boundary, vertex, and collinear cases. Cutting a convex polygon with a line follows the Sutherland-Hodgman idea: keep vertices on the desired side and insert intersection points when edges cross the cut.

Key Takeaways

- Number theory problems depend heavily on using structure instead of naive loops.
- Sieve preprocessing can support primality, factorization, totients, and divisor-related data.
- Modular arithmetic is not ordinary arithmetic with smaller numbers; division requires special handling.
- Geometry solutions need both formulas and numerical discipline around floating-point comparisons.
- Polygon algorithms are mostly careful iteration over ordered vertices plus cross products and intersections.

CP4 Reading Connections

- CP4 Section 5.3, *Number Theory*, especially Sections 5.3.1, 5.3.3, 5.3.5, 5.3.6, and 5.3.9.
- CP4 Section 7.2, *Basic Geometry Objects with Libraries*, especially Sections 7.2.1-7.2.5.
- CP4 Section 7.3, *Algorithms on Polygon with Libraries*, especially Sections 7.3.1, 7.3.3, 7.3.4, 7.3.5, and 7.3.6.
- CP4 Section 1.3.3 for checking whether mathematical or geometric preprocessing fits the input constraints.